Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
GRADUATION THESIS

Разработка веб-приложения для онлайн-бирж проектов

Обучающийся / Student Ачарья Никеш
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34211
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Нейротехнологии и программирование 2020
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Руководитель ВКР/ Thesis supervisor Логинов Иван Павлович, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

Обучающийся/Student

Документ подписан
Ачарья Никеш
20.05.2024

(эл. подпись/ signature)

Ачарья Никеш
(Фамилия И.О./ name and surname)

Руководитель ВКР/ Thesis supervisor

Документ подписан
Логинов Иван Павлович
19.05.2024

(эл. подпись/ signature)

Логинов Иван Павлович
(Фамилия И.О./ name and surname)

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ / OBJECTIVES FOR A GRADUATION THESIS

Обучающийся / Student Ачарья Никеш
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34211
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Нейротехнологии и программирование 2020
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Разработка веб-приложения для онлайн-бирж проектов
Руководитель ВКР/ Thesis supervisor Логинов Иван Павлович, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

Характеристика темы ВКР / Description of thesis subject (topic)

Тема в области фундаментальных исследований / Subject of fundamental research: нет / not
Тема в области прикладных исследований / Subject of applied research: да / yes

Основные вопросы, подлежащие разработке / Key issues to be analyzed
1. Обзор предметной области и аналогичных проектов.
2. Разработка требований к разрабатываемой системе.
3. Разработка программной архитектуры веб-приложения.
4. Проектирование пользовательского интерфейса.
5. Программная реализация веб-приложения.
6. Тестирование веб-приложения.

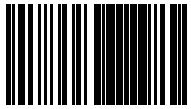
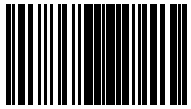
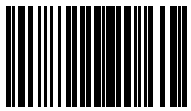
Форма представления материалов ВКР / Format(s) of thesis materials:
Пояснительная записка, презентация

Дата выдачи задания / Assignment issued on: 22.02.2024

Срок представления готовой ВКР / Deadline for final edition of the thesis 30.05.2024

СОГЛАСОВАНО / AGREED:

Руководитель ВКР/	Документ	

Thesis supervisor	подписан Логинов Иван Павлович 29.04.2024 (эл. подпись)		Логинов Иван Павлович
Задание принял к исполнению/ Objectives assumed BY	Документ подписан Ачарья Никеш 29.04.2024 (эл. подпись)		Ачарья Никеш
Руководитель ОП/ Head of educational program	Документ подписан Лисицына Любовь Сергеевна 29.04.2024 (эл. подпись)		Лисицына Любовь Сергеевна

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

АННОТАЦИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ SUMMARY OF A GRADUATION THESIS

Обучающийся / Student Ачарья Никеш
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34211
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Нейротехнологии и программирование 2020
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Разработка веб-приложения для онлайн-бирж проектов
Руководитель ВКР/ Thesis supervisor Логинов Иван Павлович, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

ХАРАКТЕРИСТИКА ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ DESCRIPTION OF THE GRADUATION THESIS

Цель исследования / Research goal
Создание проектной биржи за счет реализации веб-приложения, предоставляющего возможности по размещению и поиску сведений о проектах, исполнителях.
Задачи, решаемые в ВКР / Research tasks
Обзор предметной области и аналогичных проектов. Разработка требований к разрабатываемой системе. Разработка программной архитектуры веб-приложения. Проектирование пользовательского интерфейса. Программная реализация веб-приложения. Тестирование веб-приложения.
Краткая характеристика полученных результатов / Short summary of results/findings
Разработанное веб-приложение, выполняющее функции онлайн-биржи для выполнения проектов является полноценным решением для поиска квалифицированных работников. Он оптимизирует затраты, позволяя клиентам самим определять стоимость своего проекта. Он экономит время и позволяет выбрать наиболее квалифицированного работника для проекта.

Обучающийся/Student

Документ подписан
Ачарья Никеш
20.05.2024

(эл. подпись/ signature)

Ачарья Никеш
(Фамилия И.О./ name

and surname)

Руководитель ВКР/
Thesis supervisor

Документ подписан
Логинов Иван Павлович
19.05.2024

(эл. подпись/ signature)

Логинов Иван Павлович

(Фамилия И.О./ name and surname)

Содержание

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 7
ВВЕДЕНИЕ 8
1 Анализ предметной области и существующих программных средств 10
1.1 Исследование существующих программных средств 10
1.2 Разработка требований к обслуживанию 14
2 Разработка требований к разрабатываемой системе 19
3 Разработка программной архитектуры веб-приложения 22
3.1 Разработка архитектуры модели данных 22
3.2 Анализ инструментальных средств 25
3.2.1 Серверная часть приложения 25
3.2.2 База данных 28
4 Проектирование пользовательского интерфейса 35
5 Программная реализация веб-приложения 36
5.1 Система чата 36
5.2 Система кошелька 52
5.3 Динамическое управление блогами 55
5.4 Система управления вопросами 58
6 Тестирование веб-приложения 66
6.1 Кросс-браузерное тестирование 66
6.2 Ручное тестирование 66
ЗАКЛЮЧЕНИЕ 68
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 69

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

UI/UX (от англ. Application Programming Interface) – программный интерфейс приложения

CRUD (от англ. Create, Read, Update, Delete) – операции над данными (создание, чтение, обновление, удаление)

HTTP (от англ. Hypertext Transfer Protocol) – протокол передачи гипертекста REST (от англ. Representational State Transfer) - архитектурный стиль для разработки веб-сервисов

CSS (от англ. Cascading Style Sheets) – формальный язык декодирования и описания внешнего вида документа

ИИ - Искусственный интеллект

API (от англ. Application programming interface)– программный интерфейс приложения

CMS (от англ. Content management system) – Система управления контентом

ВВЕДЕНИЕ

Целью данной выпускной квалификационной работы было создание платформы, которая бы предоставляла функционал, аналогичный популярным сервисам, таким как Profi.ru, Upwork и Fiverr.

Цель этой платформы — предоставить пользователям, как клиентам, так и фрилансерам, удобные инструменты для эффективного общения и сотрудничества. В ходе работы был проведён глубокий анализ рынка и конкурентов, что позволило определить основные функции, необходимые для обеспечения жизнеспособности платформы на рынке.

Была создана универсальная система, которая включает в себя вход и регистрацию пользователей, систему управления контентом, интеграцию с платежным шлюзом, систему чата для облегчения общения между клиентами и фрилансерами.

Особое внимание было уделено обеспечению безопасности и удобству использования платформы. Для разработки платформы были использованы передовые технологии, включая платформу Laravel 11, выбранную благодаря её способности эффективно управлять системой чата без необходимости интеграции с WebSocket.

В процессе разработки была тщательно спроектирована надежная архитектура программного обеспечения, создан интуитивно понятный пользовательский интерфейс, проведены обширные этапы тестирования для обеспечения функциональности платформы, а также выявлены и исправлены многочисленные ошибки.

Отдавая приоритет безопасности(5), прозрачности и прибыльности, платформа гарантирует, что все участники смогут участвовать в честных и эффективных транзакциях.

Таким образом, усовершенствованная система предлагает надежную и безопасную среду для общения между клиентами и независимыми работниками, повышая их вовлеченность и общую удовлетворенность

клиентов. Это делает ее очень привлекательным вариантом для пользователей, ищущих эффективные решения.

Проект предоставляет следующие основные возможности:

1) система входа и регистрации, обслуживающая администраторов, фрилансеров и клиентов, позволяющая им легко получить доступ к своим учетным записям;
2) система CRUD для управления контентом, позволяющая пользователям создавать, читать, обновлять и удалять различные типы контента;
3) система кошельков для удобной оплаты заказов, а также интеграция с платежным шлюзом с использованием API для бесшовных транзакций;
4) система чата, способствующая эффективному общению между пользователями, повышению совместной работы и производительности.

1 Анализ предметной области и существующих программных средств

1.1 Исследование существующих программных средств

Фриланс-площадки, такие как Upwork, Fiverr и Freelancer(11), связывают компании и частных клиентов с фрилансерами. Клиенты или компании составляют техническую задачу, публикуют проекты, а фрилансеры делают на них ставки и берутся за работу, если их выбирают. После выбора фрилансера работа выполняется и оплачивается через платформу. Эти сайты отличаются гибкостью, позволяя компаниям быстро находить квалифицированных работников, а фрилансерам - выбирать из множества вакансий. Они создают возможности для удаленной работы, но также сопряжены с такими проблемами, как обеспечение справедливой оплаты и гарантий занятости. В целом, платформы для фриланса упрощают совместную работу людей из любой точки мира. Однако в любом случае они не решают все возможные проблемы, но в последнее время добавляют новые технологии контроля, проверки ведения проектов и системы наказания фрилансеров и клиентов.

Upwork(2) — это американская платформа для фриланса, основанная в 2015 году. На сегодняшний день она является самой популярной и крупной платформой во всем мире, имеет много уникального функционала и занимается привлечением, как клиентов, так и фрилансеров. Она объединяет бизнес с фрилансерами из различных областей, таких как писательство, звукорежиссура, моушен-дизайн, 3д-дизайн, создание эффектов и анимаций, копирайтинг, аналитика данных, разработка ИИ-продуктов, программирование, маркетинг и многое другое. Она предлагает ряд инструментов для управления проектами, отслеживания рабочего времени и безопасных платежей, что облегчает клиентам и фрилансерам эффективное сотрудничество. Она также включает в себя такие функции, как подбор талантов, где передовые алгоритмы помогают клиентам найти наиболее

подходящих фрилансеров, и фильтры, помогающие пользователям найти идеального кандидата, соответствующего требованиям их проекта. Там реализованы очень продуманные алгоритмы сортировки и составления предложений. Платформа обеспечивает безопасные и своевременные платежи через систему условного депонирования, которая удерживает средства до тех пор, пока обе стороны не будут удовлетворены выполненной работой. На платформе зарегистрировано более 20 миллионов фрилансеров и более 5 миллионов зарегистрированных клиентов. Это является самым большим показателем среди всех конкурентов, за счет чего площадка по праву является лидером на рынке предоставления фриланс услуг. На платформе насчитывается около 827 000 активных клиентов, которые воспользовались сервисом за последний год (на основе технического отчета и DemandSage). У приложения 172 тыс. отзывов в Play Store, в среднем 4,1 из 5 возможных, и 2,2 тыс. отзывов в AppStore, в среднем 4,6 из 5 возможных.

Fiverr(3) - еще одна очень крупная платформа для фриланса, созданная в 2010 году. Глобальный рейтинг веб-сайтов по версии similarweb у платформы составляет 837 место. Эта платформа в основном ориентирована на IT-сектор. Фрилансеры рекламируют свои услуги, такие как веб-дизайн, разработка веб-сайтов, веб-приложений, телеграмм-чатов, телеграмм-приложений, десктопных приложений, кроссплатформенных решений, интеграция с машинным обучением и многое другое. Клиент имеет право выбрать любого исполнителя на свое усмотрение, посмотрев отзывы, выполненные работы и просто пообщавшись с фрилансерами.

К минусам Bring можно отнести перегруженный дизайн интерфейса.

Profi – Profi ru(4), основанный в 2005 году, является ведущей платформой для фриланса в Российской Федерации и странах СНГ. Платформа объединяет клиентов (заказчиков) и фрилансеров (исполнителей). Ознакомившись с последними данными, видно, что в странах СНГ и Российской Федерации насчитывается уже более 15 миллионов пользователей

на площадке. Цель этой платформы - обеспечить быструю связь между клиентами и фрилансерами. Области фриланса не ограничены, там есть специалисты из любой сферы - починка техники, строители, монтажники, сантехники, специалисты по ремонту ноутбуков, программисты, маркетологи, дизайнеры и многие другие. Эта платформа предоставляет услуги фриланса как онлайн, так и офлайн. Клиенты могут легко выбирать время и место для встречи с клиентами и предоставления им работы. Это не только помогает продвигать фрилансеров, но и ускоряет выполнение работы, поскольку имеет систему отзывов, благодаря которой клиенты могут легко доверять тем фрилансерам, которые имеют высокие показатели. Платформа доступна как на Android, так и на iOS. Пользовательский интерфейс прост и отзывчив. Платформа не просто так занимает лидирующую позицию на рынке предоставления услуг, так как сделана очень качественно и решает большинство возникающих проблем на рынках Российской Федерации и странах СНГ.

Теперь необходимо произвести сравнительную характеристику функциональных возможностей вышепредставленный приложений. Ниже представлена таблица в которой выписаны основные возможности этих приложений (Таблица 1).

Таблица 1 –Сравнение возможностей приложений конкурентов

Функциональность	**Fiverr**	**Upwork**	**Profi**
Проверка профиля как клиентов, так и фрилансеров	+	+	-
Система условного депонирования для хранения денежных средств клиентов	+	+	-
Система коммуникации между клиентами и фрилансерами	+	+	+
Система отзывов клиентов и фрилансеров	+	+	+
Корпоративные решения	-	+	-
Система входа в систему/регистрация	+	+	+
Интеграция платежных систем	+	+	+
Система кошельков	-	-	+
Функция почасовой оплаты	-	+	-

Анализируя Upwork, Fiverr и Profi ru можно обнаружить такие ключевые особенности, как показано выше в таблице. Основываясь на ней, можно сравнить Fiverr, Upwork и Profi. И Fiverr, и Upwork предлагают функции для проверки профиля клиентов и фрилансеров, но в Profi такой функции нет.

Кроме того, и в Upwork, и в Fiverr есть система условного депонирования для хранения средств клиентов, которой нет в Profi. На каждом фриланс-сайте есть система чата и система отзывов клиентов и фрилансеров, но только на Upwork есть корпоративное решение и функция почасовой оплаты. Все фриланс площадки имеют систему входа в систему и регистрации. Пользовательский интерфейс у всех площадок сильно отличается, однако везде он выполнен в профессиональных цветах и минимализме, однако дизайн Profi ru немного староват.

1.2 **Разработка требований к обслуживанию**

Приведенный выше анализ всех трех веб-сайтов помог определить функциональные возможности, которые необходимо включить в новый проект и быть конкурентоспособным. Ниже приведены все функциональные возможности, которые необходимо использовать.

Система входа/регистрации: Данная система предоставляет возможность входа и регистрации для администраторов, фрилансеров и клиентов. Администраторы авторизуются в системе на основе сеанса, что позволяет им управлять функциональностью и данными. Для клиентов и фрилансеров используется API(8). При входе в систему они получают токен, который используется для аутентификации их запросов к API. Этот токен обычно включается в заголовки запросов и проверяется API(8) для авторизации доступа к запрашиваемым ресурсам.

Система CRUD: Внедрена система CRUD (Создавать, читать, обновлять, удалять) в админ-панели, чтобы упростить управление контентом блога и категориями. Можно редактировать любые посты и управлять интерфейсом.

Система кошельков для оплаты заказов. В системе фрилансер может пополнить свой счет деньгами, которые он может использовать для подачи заявки на написание сообщения клиенту. Чтобы отправить первое сообщение, фрилансеру необходимо внести небольшую плату со своего счета. Это помогает убедиться в серьезности его намерений. Однако, если клиент не

прочитает сообщение в течение 7 дней, гонорар возвращается на счет фрилансера. Система кошельков упрощает фрилансерам управление своими средствами, которые используются для поиска работы.

Интеграция платежного шлюза с помощью API: Платежная система используется для пополнения кошелька фрилансеров. Это означает, что фрилансеры могут использовать платежную систему для пополнения своего счета, с которого они затем могут отправить первое сообщение клиенту. Платежная система играет решающую роль в создании честной и эффективной торговой площадки для общения и совместной работы фрилансеров и клиентов. Она обеспечивает безопасность, прозрачность и выгодность транзакций для всех участников.

Система чата: Система чата(1) между фрилансерами и клиентами предназначена для облегчения эффективного общения и совместной работы. Как только клиент разместил вакансию, а фрилансер подал заявку (фрилансеру необходимо заплатить небольшую сумму, установленную администратором), после оплаты фрилансером они могут общаться напрямую через нашу систему обмена сообщениями. Система чата позволяет обмениваться сообщениями в режиме реального времени, позволяя фрилансерам и клиентам обсуждать детали проекта, уточнять требования, согласовывать условия и задавать вопросы. Такая прямая и мгновенная коммуникация упрощает процесс найма, позволяя обеим сторонам быстро достигать договоренностей и принимать решения. В данной системе чата мы использовали функцию Laravel Broadcasting, которая позволяет вам транслировать события на ваш сервер Laravel Echo, который затем транслирует их в клиентские приложения, где мгновенно передаются данные с сервера клиенту. Laravel Echo Laravel Echo - это библиотека JavaScript, которая позволяет легко подписываться на каналы и прослушивать события, транслируемые Laravel(10). Он предоставляет простой API для подключения к серверам WebSocket, подписки на каналы и составления списка событий. Laravel Reberb – это сервер WebSocket с

расширенным протоколом для трансляции событий. Это позволяет настроить связь в режиме реального времени в данной системе.

Настройки профиля фрилансера и клиента: Настройка профиля фрилансера на платформе позволяет фрилансерам создавать всесторонний и профессиональный профиль, чтобы продемонстрировать свои навыки, опыт, квалификацию и бюджет. Фрилансер может ввести информацию о своем профиле, такую как его имя, местоположение и фотографию профиля и отправить профессиональное резюме, описывающее его опыт работы. Они могут перечислить ключевые навыки, предоставить подробную информацию о своем опыте работы, образовании и загрузить образцы в свое портфолио. Они должны добавить свой идентификатор, который предоставляется правительством, на основании которого администратор проверяет профиль фрилансера, например паспорт или удостоверение личности. Настройка профиля в системе очень проста. Клиентам необходимо только подтвердить свой номер телефона, им не нужно добавлять дополнительные сведения о себе.

Гость может добавить информацию о своих проектах. Регистрация осуществляется по номеру телефона. Он не может разместить проект в системе до тех пор, пока не зарегистрируется в системе. После успешной регистрации информация, включающая требования клиентов, такие как цена, местоположение и крайний срок, сохраняется в базе данных. Зарегистрированными пользователями являются клиенты и фрилансеры.

Система управления вопросами находится под контролем администратора. Администратор добавляет вопросы в каждую категорию.

На рисунке 1 представлена диаграмма прецедентов со всеми описанными ролями пользователей.

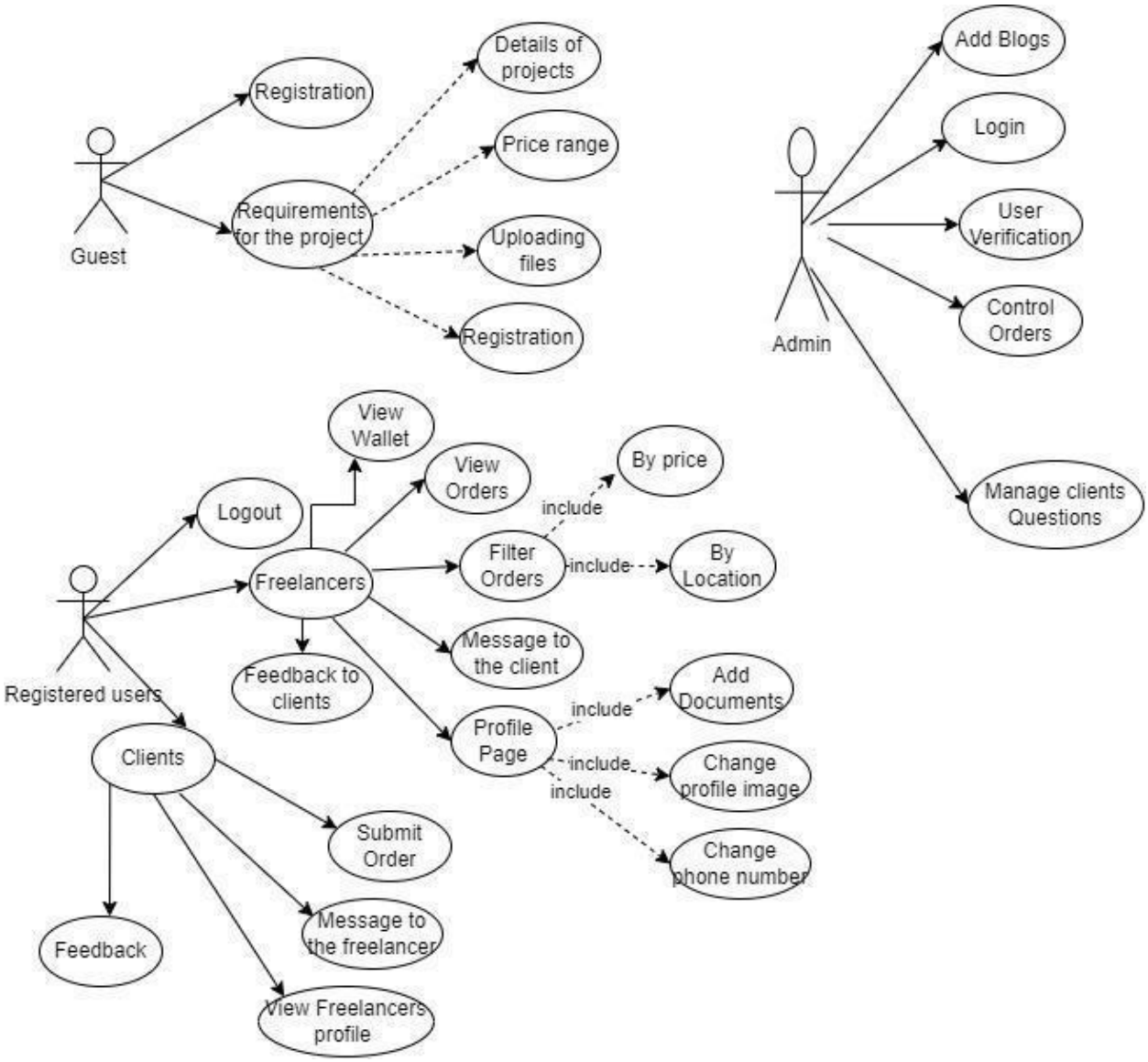


Рисунок 1 –Диаграмма прецедентов

Система управления вопросами: В системе администратор может добавлять вопросы, основанные на различных категориях. Клиенты могут искать данные и добавлять свои требования, основываясь на этом модуле вопросов. Например, если категорией является веб-дизайн, администратор может задать список примеров-вопросов. Клиенты могут добавить свои требования, указать сроки и предоставить другие сведения на основе этих

вопросов, а затем опубликовать вакансию. Кроме того, клиенты могут просмотреть список специальностей при добавлении своих требований.

Динамический обзор на главной странице: Отзывы, отображаемые на главной странице веб-сайта, являются динамическими. Клиенты пишут отзывы фрилансерам, и эти отзывы отображаются на главной странице веб-сайта в динамическом режиме.

Зарегистрированные пользователи:

- клиенты,
- фрилансеры,
- администраторы.

Клиенты публикуют свои проекты и ждут, пока фрилансеры отправят им сообщение о сотрудничестве. Как только фрилансер отправит сообщение клиенту, он сможет открыть и проверить его профиль.

У фрилансеров есть страница для просмотра всех заказов. В настройках профиля фрилансеры могут изменить название своей должности, опыт работы, местоположение, загрузить фотографию профиля, изменить номер телефона и удалить свой профиль. Проекты отображаются на экране в соответствии с их должностью и местоположением. Чтобы отправить сообщение клиенту, они должны заплатить определенную сумму денег, которой управляет администратор из панели администратора. фрилансеры могут просматривать свой кошелек, а также пополнять его для оплаты обмена сообщениями с клиентом. Есть возможность фильтровать заказы по цене и местоположению.

2 Разработка требований к разрабатываемой системе

В контексте данного проекта функции, которые в обязательном порядке требуются системе для выполнения задачи как для клиента, так и для фрилансера, являются функциональными требованиями.Управлять системой должны не только клиенты и фрилансеры, но и администратор. Ниже приведены некоторые из необходимых функций системы.

Регистрация и авторизация: как клиенты, так и фрилансеры могут зарегистрироваться на веб-сайте и при необходимости изменить данные своего профиля. Кроме того, существует вероятность того, что они забудут пароль. Таким образом, есть функция смены пароля.

Профиль пользователей и фрилансеров: они оба могут просматривать профиль друг друга. Кроме того, можно загружать фотографии профиля. Они оба могут просматривать отзывы друг о друге.

Поиск и фильтрация: клиенты могут осуществлять поиск с помощью рейтингов фрилансеров, в то время как фрилансеры ищут проекты в соответствии со своей должностью. Веб-дизайнер может видеть на веб-странице только задание веб-дизайнера. Кроме того, фрилансер может фильтровать проекты по местоположению и цене.

Обмениваться сообщениями: система чата для общения фрилансеров и клиентов друг с другом в режиме реального времени. Фрилансеры должны внести небольшую плату за отправку сообщения клиенту.

Операции по оплате и финансированию: система кошельков внутри веб-приложения играет важную роль в управлении финансовыми операциями. Платежи для отправки клиентам фрилансера осуществляются с помощью платежного шлюза. Если клиент не видит сообщение в течение 7 дней, платеж, внесенный фрилансерами, автоматически возвращается на кошелек.

Администрация: система управления вопросами полностью контролируется администратором. Кроме того, содержимое веб-приложения,

такое как блоги, политики конфиденциальности, чат центра поддержки, находится под контролем администратора.

Сценарий создания нового проекта: клиент выполняет поиск и нажимает на кнопку поиска. Задаются подробные вопросы о проектах, включая бюджет, дату, файлы, удобное место для встречи и многое другое. Клиент не может опубликовать проект до тех пор, пока не зарегистрируется в системе. Итак, он должен в конце ввести номер телефона для регистрации. После регистрации клиент теперь может опубликовать проект.

На этом этапе был выполнен эскиз или, выражаясь техническим языком, дизайн веб-сайта. Это было сделано в figma. Дизайн отражает веб-сайт. Это помогает визуализировать внешний вид веб-сайта. Согласно когнитивной психологии, цвет, используемый в различных областях веб-сайта, включая кнопки, панель навигации и цвет системы чата, играет жизненно важную роль в том, чтобы сделать веб-сайт более удобным для пользователя и отзывчивым. Дизайн был разработан как для мобильных устройств, так и для настольных компьютеров.

Figma – это веб-приложение, используемое UI/UX дизайнерами для разработки веб-сайтов. Оно было создано в 2012 году с целью сделать процесс проектирования более плавным и эффективным. В обновленной версии также появились новые функции. Например, если мы отправим скриншот любого веб-сайта, искусственный интеллект создаст для нас такой же дизайн.

На рисунке 2 изображен предполагаемый дизайн сайта.

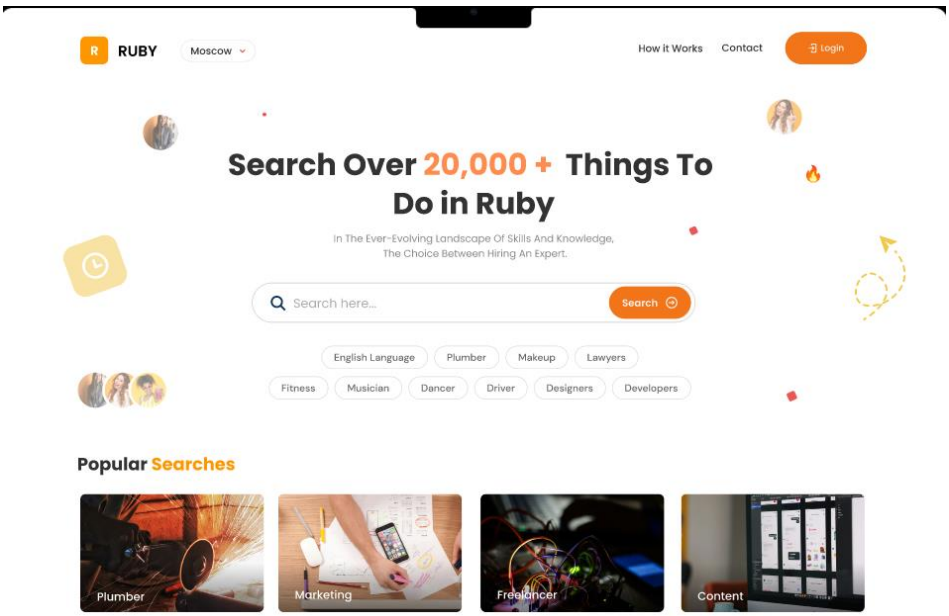


Рисунок 2 –Дизайн сайта

3 Разработка программной архитектуры веб-приложения

3.1 Разработка архитектуры модели данных

На данном этапе была разработана логическая модель данных базы данных сервиса (рисунок 3). Эта модель прекрасно описывает основные сущности базы данных и взаимосвязи между ними. Поля всех сущностей логической модели данных также были подробно описаны с указанием их типов данных и кратким описанием.



Рисунок 3 –Логическая модель

id (varchar(255), NN):

- описание: уникальный идентификатор для каждого сеанса;
- тип: переменная символьная строка максимальной длиной 255 символов;
- ограничения: не null (NN), что означает, что это поле должно содержать значение.

user_id (bigint):

- описание: уникальный идентификатор пользователя, связанный с сеансом;
- тип: большое целое число, подходит для хранения больших числовых значений;
- назначение: привязывает сеанс к определенному пользователю в таблице users, облегчая отслеживание пользователей и управление ими.

ip_address (varchar(45)):

- описание: IP-адрес, с которого был инициирован сеанс;
- тип: переменная символьная строка максимальной длиной 45 символов, включающая адреса как IPv4, так и IPv6;
- назначение: помогает регистрировать и отслеживать активность пользователей на основе их IP-адреса.

user_agent (текст):

- описание: информация о браузере пользователя и операционной системе;
- тип: текст, позволяющий хранить длинные строки;

- назначение: предоставляет информацию о клиентской среде, полезную для отладки и оптимизации взаимодействия с пользователем.

last_activity (int, NN):

- описание: временная метка последнего действия во время сеанса;
- тип: целое число;
- ограничения: не null(nn), что означает, что это поле должно содержать значение;
- назначение: используется для отслеживания времени последнего взаимодействия, необходимого для управления сеансом и таймаутов.

Идентификатор (bigint, NN):

- описание: уникальный идентификатор для каждой категории;
- тип: большое целое число;
- ограничения: не null(nn), что означает, что это поле должно содержать значение.

name (varchar(255)):

- описание: название категории;
- тип: переменная символьная строка максимальной длиной 255 символов;
- назначение: отображает название категории.

slug (varchar(255)):

- описание: URL-адрес или путь к изображению, представляющему категорию;

- тип: переменная символьная строка максимальной длиной 255 символов;

- назначение: позволяет визуально представить категорию.

meta_title (varchar(255).

3.2 **Анализ инструментальных средств**

3.2.1 Серверная часть приложения

В нашей системной архитектуре Laravel(10) служит основой для обработки внутренней логики, легко взаимодействуя с нашей базой данных SQL для управления хранением и извлечением данных. Экосистема пользователей включает в себя три различные роли: администраторов, клиентов и фрилансеров. Администраторы осуществляют общий контроль за работой всей системы. Клиенты могут публиковать объявления о вакансиях, а фрилансеры взаимодействуют с клиентами с помощью функций обмена сообщениями.

Функциональные возможности управления клиентами позволяют администраторам контролировать взаимодействие с клиентами и поддерживать с ними прочные отношения. В то же время возможности управления CMS позволяют создавать и публиковать динамичный контент, гарантируя, что наша платформа остается динамичной и привлекательной. Функции управления фрилансерами упрощают процесс адаптации фрилансеров и отслеживания их эффективности, оптимизируя процессы привлечения и удержания талантов.

Функции управления местоположением облегчают геопространственный анализ и локализацию, гарантируя, что наша платформа эффективно обслуживает различные географические рынки. Функции управления категориями обеспечивают детальный контроль над классификациями должностей и предлагаемых услуг, повышая доступность и актуальность для пользователей.

При взаимодействии с внешним интерфейсом клиенты и фрилансеры легко взаимодействуют с нашей системой посредством HTTP-запросов, отправляемых через наш Laravel API. Используя Laravel Eloquent, наша система эффективно выполняет запросы к базе данных, обеспечивая быстрый поиск данных и манипулирование ими для выполнения запросов пользователей. Эта парадигма распространяется и на CRUD-операции, обеспечивая согласованность и эффективность в задачах управления данными.

Однако в системе чата используется особый подход к обмену сообщениями в режиме реального времени между фрилансерами и клиентами. Используя WebSockets(7) и Laravel Echo, сообщения передаются мгновенно, что обеспечивает пользователям бесперебойную коммуникацию. Pusher и Reverb еще больше расширяют эту функциональность, обеспечивая надежную доставку сообщений и синхронизацию между устройствами. Кроме того, наша система использует протокол SMTP для доставки электронных писем, связанных с транзакциями, что повышает вовлеченность пользователей и облегчает получение важных уведомлений и обновлений.

В этих проектах используются следующие языки программирования :-

Vue: В системе мы использовали Vue.js для реализации пользовательского интерфейса и добавления динамичности в сайт. Vue.js это прогрессивный JavaScript-фреймворк, используемый для создания пользовательских интерфейсов, в частности одностраничных приложений. Он разработан с возможностью постепенного внедрения, что означает, что можно использовать такие модули из фреймворка, какие нужны для конкретной задачи. Vue.js известен своей простотой, гибкостью и легкостью интеграции с другими библиотеками или существующими проектами. В нем используется система реактивной привязки данных и компонентная архитектура, что делает его эффективным для управления динамическими аспектами современных веб-приложений. Доступная базовая библиотека Vue, наряду с

поддерживающими ее библиотеками и инструментами, обеспечивает полноценную экосистему для разработки многофункциональных и отзывчивых интерфейсных приложений.

PHP: В системе используется Laravel версии 11 - фреймворк написанный на языке PHP для разработки API и серверной части. Laravel - это мощный PHP-фреймворк, используемый для разработки API и серверных систем. Он предлагает элегантный синтаксис и надежные инструменты для таких задач, как маршрутизация, аутентификация и кэширование. Встроенные функции Laravel, такие как Eloquent ORM для взаимодействия с базами данных и обширная библиотека готовых компонентов, упрощают разработку. Поддержка RESTful API делает его отличным выбором для создания масштабируемых и поддерживаемых серверных сервисов.

CSS: В системе мы использовали Sass и Tailwind Css вместе для стилизации. SASS (Syntactically Awesome Stylesheets) - это мощный язык расширения CSS, который расширяет возможности традиционного CSS. Он предоставляет такие функции, как переменные, вложенность, микширование и наследование, которые делают таблицы стилей более удобными в обслуживании и упрощают управление ими. С помощью SASS разработчики могут писать более чистый и упорядоченный CSS-код, сокращая избыточность и повышая эффективность при оформлении веб-приложений.Tailwind CSS - это CSS-фреймворк, который предоставляет набор предварительно разработанных служебных классов для оформления веб-интерфейсов. В отличие от традиционных CSS-фреймворков, которые поставляются с предопределенными компонентами, Tailwind CSS предлагает низкоуровневые служебные классы, которые позволяют разработчикам быстро создавать пользовательские проекты без написания пользовательского CSS. Применяя служебные классы непосредственно в HTML-разметке, разработчики могут быстро создавать прототипы и выполнять итерации над

проектами, что приводит к созданию более гибких и удобных в обслуживании кодовых баз.

3.2.2 База данных

MySQL известна как система управления базами данных с открытым исходным кодом, лицензируемая по лицензии GNU GPL, что позволяет разработчикам свободно использовать и модифицировать ее. Она также предлагает поддержку нескольких языков программирования, таких как PHP, Java, Python и Perl. Кроме того, MySQL хорошо оптимизирован для быстрой и эффективной обработки больших объемов данных, что делает его надежным выбором для многих приложений. Она также предлагает высокий уровень защиты данных и масштабируемости, что позволяет использовать его для широкого спектра проектов, от небольших проектов до крупных корпоративных систем. В целом, MySQL — универсальный и популярный выбор для разработчиков и компаний по всему миру благодаря своей производительности, надежности и открытости.

Несколько исторических фактов:

- 2003: запуск WordPress. MySQL используется для хранения и управления данными, что позволяет миллионам веб-сайтов работать стабильно и эффективно;
- 2003: запуск Google AdSense. MySQL обеспечивает высокую производительность и масштабируемость для обработки данных о кликах и показах рекламы;
- 2004: Facebook начинает активно использовать MySQL. Оптимизация и доработка MySQL позволили поддерживать работу социальной сети при огромном росте числа пользователей;

- 2008: Приобретение MySQL компанией Sun Microsystems за $1 миллиард. Это позволило MySQL получить больше ресурсов и ускорить разработку;
- 2010: Oracle приобретает Sun Microsystems, включая MySQL. Oracle продолжает поддерживать и развивать MySQL, выпуская регулярные обновления, несмотря на первоначальные опасения сообщества.

MySQL используется в этом проекте из-за его надежности, простоты использования и экономической эффективности. Это база данных с открытым исходным кодом, известна своей стабильностью и высокой производительностью даже при больших нагрузках. MySQL также обладает высокой масштабируемостью, что позволяет ей расти в соответствии с потребностями проекта. Ее широкая совместимость с различными языками программирования и платформами в сочетании с широкой поддержкой сообщества делает его универсальным и хорошо поддерживаемым выбором. Кроме того, MySQL предлагает надежные функции безопасности для обеспечения защиты данных, что делает его идеальным решением для эффективного управления данными проекта.

MySQL помогает эффективно и бесперебойно выполнять проект, предоставляя надежную и стабильную систему баз данных, обеспечивающую стабильную производительность. Ее удобная настройка и управление упрощают работу с базой данных, сокращая время и усилия разработчиков. Быстрая обработка запросов MySQL повышает общую эффективность обработки данных, что приводит к более быстрым ответам приложений. Масштабируемость MySQL позволяет ему адаптироваться к увеличивающимся объемам данных по мере роста проекта, обеспечивая долгосрочную жизнеспособность. Кроме того, его надежные функции безопасности защищают целостность данных и конфиденциальность, способствуя бесперебойной и безопасной работе проекта.

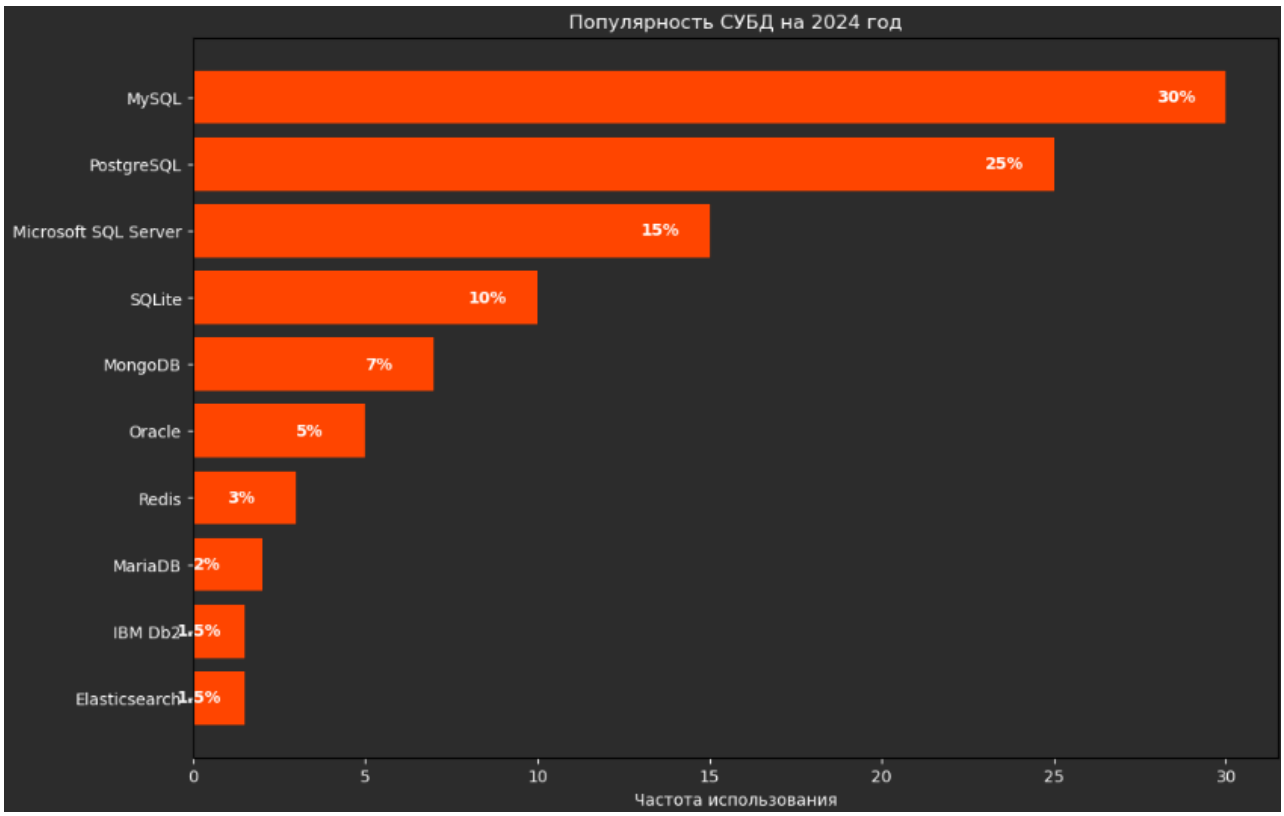


Рисунок 4 –10 наиболее популярных СУБД по данным stackoverflow.com

4 Проектирование пользовательского интерфейса

Разработка макетов и прототипов

Цвет для всей веб-страницы выбран белый, чтобы сохранить простоту. Цвет кнопки - оранжевый, что выглядит более привлекательно. Дизайн разработан как для мобильной, так и для настольной версии. На главной странице веб-сайта есть строка поиска. Строка поиска размещена таким образом, чтобы она выглядела привлекательно и дружелюбно для пользователей. Основная часть дизайна состоит из блогов, отзывов клиентов о фрилансерах, о том, как это работает, о проекте и многом другом.

Тестирование прототипа проводилось путем перехода с одной страницы на другую, поиска всей информации о веб-сайте, проверки правильности расположения кнопок на figma, выявления других проблем и их сортировки.

Также с помощью adobe photoshop были скорректированы высота, ширина, форматы изображений и размещены в нужном месте, чтобы интерфейс выглядел более привлекательно. Страницы с отзывами представлена на рисунке 5.

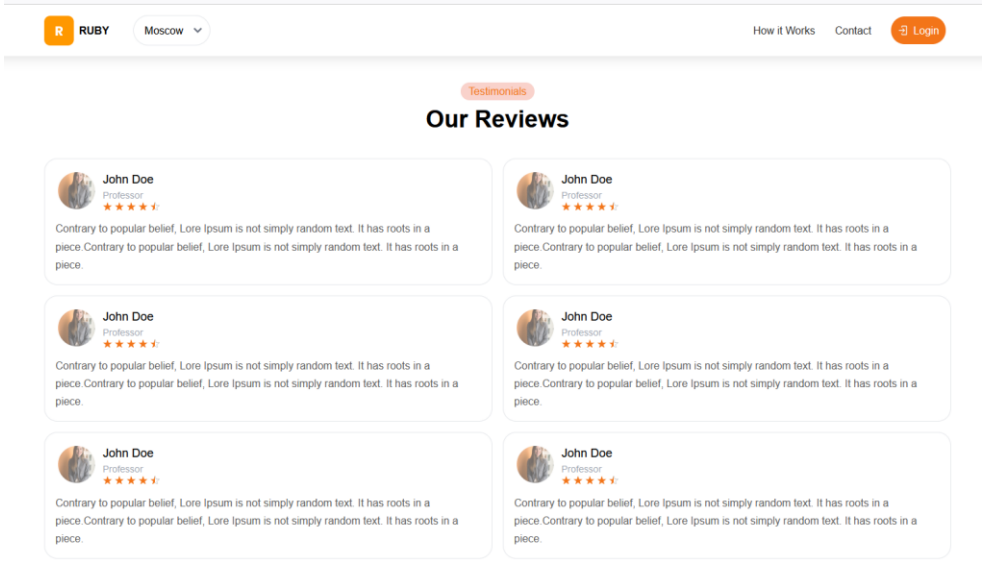


Рисунок 5 –Отзывы клиентов о фрилансерах

5 Программная реализация веб-приложения

5.1 Система чата(11)

```
 /**
     * Mark all notifications as read.
     *
     * @return void
     */
    public function markAsRead()
    {
        $this->each->markAsRead();
    }

    /**
     * Mark all notifications as unread.
     *
     * @return void
     */
    public function markAsUnread()
    {
        $this->each->markAsUnread();
    }
<?php

namespace App\Services;

use App\Models\ChMessage as Message;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Facades\Auth;
use Exception;

class Messenger
{
    /**
```

```
     * Fetch & parse message and return the message card
     * view as a response.
     *
     * @param Message $prefetchedMessage
     * @param int $id
     * @return array
     */
    public function parseMessage($prefetchedMessage = null, $id
= null)
    {
        $msg = null;
        $attachment = null;
        $attachment_type = null;
        $attachment_title = null;
        if (!!$prefetchedMessage) {
            $msg = $prefetchedMessage;
        } else {
            $msg = Message::where('id', $id)->first();
            if (!$msg) {
                return [];
            }
        }
        return [
            'id' => $msg->id,
            'from_id' => $msg->from_id,
            'to_id' => $msg->to_id,
            'message' => $msg->body,
            'timeAgo' => $msg->created_at->diffForHumans(),
            'created_at' => $msg->created_at->toIso8601String(),
            'isSender' => ($msg->from_id == Auth::user()->id),
            'seen' => $msg->seen,
        ];
    }

    /**
```

```
     * Default fetch messages query between a Sender and
Receiver.
     *
     * @param int $user_id
     * @return Message|\Illuminate\Database\Eloquent\Builder
     */
    public function fetchMessagesQuery($user_id)
    {
        return Message::where('from_id', Auth::user()->id)-
>where('to_id', $user_id)
            ->orWhere('from_id', $user_id)->where('to_id',
Auth::user()->id);
    }

        /**
     * Return a message card with the given data.
     *
     * @param Message $data
     * @param boolean $isSender
     * @return string
     */
    public function messageCard($data, $renderDefaultCard =
false)
    {
        if (!$data) {
            return '';
        }
        if($renderDefaultCard) {
            $data['isSender'] =  false;
        }
        return $data;
    }

     /**
     * Trigger an event using Pusher
```

```
     *
     * @param string $channel
     * @param string $event
     * @param array $data
     * @return void
     */
    public function push($channel, $event, $data)
    {
        // return $this->pusher->trigger($channel, $event,
$data);
    }


    /**
     * create a new message to database
     *
     * @param array $data
     * @return Message
     */
    public function newMessage($data)
    {
        $message = new Message();
        $message->from_id = $data['from_id'];
        $message->to_id = $data['to_id'];
        $message->body = $data['body'];
        $message->attachment = $data['attachment'];
        $message->save();
        return $message;
    }

    /**
     * Make messages between the sender [Auth user] and
     * the receiver [User id] as seen.
     *
     * @param int $user_id
```

```
     * @return bool
     */
    public function makeSeen($user_id)
    {
        Message::Where('from_id', $user_id)
            ->where('to_id', Auth::user()->id)
            ->where('seen', 0)
            ->update(['seen' => 1]);
        return 1;
    }

    /**
     * Get last message for a specific user
     *
     * @param int $user_id
     * @return
Message|Collection|\Illuminate\Database\Eloquent\Builder|\Illumi
nate\Database\Eloquent\Model|object|null
     */
    public function getLastMessageQuery($user_id)
    {
        return $this->fetchMessagesQuery($user_id)->latest()-
>first();
    }

    /**
     * Count Unseen messages
     *
     * @param int $user_id
     * @return Collection
     */
    public function countUnseenMessages($user_id)
    {
        return Message::where('from_id', $user_id)-
>where('to_id', Auth::user()->id)->where('seen', 0)->count();
```

```
    }

    /**
     * Get user list's item data [Contact Itme]
     * (e.g. User data, Last message, Unseen Counter...)
     *
     * @param int $messenger_id
     * @param Collection $user
     * @return string
     */
    public function getContactItem($user)
    {
        try {
            // get last message
            $lastMessage = $this->getLastMessageQuery($user-
>id);
            // Get Unseen messages counter
            $unseenCounter = $this->countUnseenMessages($user-
>id);
            if ($lastMessage) {
                $lastMessage->created_at = $lastMessage-
>created_at->toIso8601String();
                $lastMessage->timeAgo = $lastMessage-
>created_at->diffForHumans();
            }
            return [
                'get' => 'users',
                'user' => $this->getUserWithAvatar($user),
                'lastMessage' => $lastMessage,
                'unseenCounter' => $unseenCounter,
            ];
        } catch (\Throwable $th) {
            throw new Exception($th->getMessage());
        }
    }
```

```
    /**
     * Get user with avatar (formatted).
     *
     * @param Collection $user
     * @return Collection
     */
    public function getUserWithAvatar($user)
    {
        $user->avatar = 'https://www.gravatar.com/avatar/' .
md5(strtolower(trim($user->email)));
        return $user;
    }


    /**
     * Delete Conversation
     *
     * @param int $user_id
     * @return boolean
     */
    public function deleteConversation($user_id)
    {
        try {
            foreach ($this->fetchMessagesQuery($user_id)->get()
as $msg) {
                $msg->delete();
            }
            return 1;
        } catch (Exception $e) {
            return 0;
        }
    }

    /**
```

```
     * Delete message by ID
     *
     * @param int $id
     * @return boolean
     */
    public function deleteMessage($id)
    {
        try {
            $msg = Message::where('from_id', auth()->id())-
>where('id', $id)->firstOrFail();
            $msg->delete();
            return 1;
        } catch (Exception $e) {
            throw new Exception($e->getMessage());
        }
    }
}



<?php

namespace App\Http\Controllers\Message;

use App\Http\Controllers\Controller;
use App\Models\User;
use App\Services\Messenger;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Inertia\Inertia;
use App\Models\ChMessage as Message;
use Illuminate\Support\Facades\DB;

class MessagesController extends Controller
{
```

```
    protected $perPage = 30;

     /**
     * Fetch data (user, favorite.. etc).
     *
     * @param Request $request
     * @return JsonResponse
     */
    public function idFetchData(Request $request)
    {
        $messenger = new Messenger();

        $fetch = User::where('id', $request['id'])->first();
        if($fetch){
            $userAvatar = $messenger->getUserWithAvatar($fetch)-
>avatar;
        }
        return Inertia::render([
            'fetch' => $fetch ?? null,
            'user_avatar' => $userAvatar ?? null,
        ]);
    }

     /**
     * Send a message to database
     *
     * @param Request $request
     * @return JsonResponse
     */
    public function send(Request $request)
    {

        $messanger = new Messenger();
        // default variables
        $error = (object)[
```

```
            'status' => 0,
            'message' => null
        ];

        if (!$error->status) {
            $message = $messanger->newMessage([
                'from_id' => Auth::user()->id,
                'to_id' => $request['id'],
                'body' =>
htmlentities(trim($request['message']), ENT_QUOTES, 'UTF-8'),
            ]);
            $messageData = $messanger->parseMessage($message);
            if (Auth::user()->id != $request['id']) {
                $messanger->push("private-
chatify.".$request['id'], 'messaging', [
                    'from_id' => Auth::user()->id,
                    'to_id' => $request['id'],
                    'message' => $messageData
                ]);
            }
        }

        // send the response
        return Inertia::render([
            'error' => $error,
            'message' => $messageData ?? [],
            'tempID' => $request['temporaryMsgId'],
        ]);
    }

     /**
     * fetch [user/group] messages from database
     *
     * @param Request $request
     * @return JsonResponse
```

```
     */
    public function fetch(Request $request)
    {
        $messanger = new Messenger();
        $query = $messanger->fetchMessagesQuery($request['id'])->latest();
        $messages = $query->paginate($request->per_page ?? $this->perPage);
        $totalMessages = $messages->total();
        $lastPage = $messages->lastPage();
        $response = [
            'total' => $totalMessages,
            'last_page' => $lastPage,
            'last_message_id' => collect($messages->items())->last()->id ?? null,
            'messages' => '',
        ];

        // if there is no messages yet.
        if ($totalMessages < 1) {
            $response['messages'] ='<p class="message-hint center-el"><span>Say \'hi\' and start messaging</span></p>';
            return Inertia::render($response);
        }
        if (count($messages->items()) < 1) {
            $response['messages'] = '';
            return Inertia::render($response);
        }
        $allMessages = null;
        foreach ($messages->reverse() as $message) {
            $allMessages .= $messanger->messageCard(
                $messanger->parseMessage($message)
            );
        }
        $response['messages'] = $allMessages;
```

```
        return Inertia::render($response);
    }

     /**
     * Make messages as seen
     *
     * @param Request $request
     * @return JsonResponse|void
     */
    public function seen(Request $request)
    {
        // make as seen
        $messanger = new Messenger();
        $seen = $messanger->makeSeen($request['id']);
        // send the response
        return Inertia::render([
            'status' => $seen,
        ]);
    }

     /**
     * Get contacts list
     *
     * @param Request $request
     * @return JsonResponse
     */
    public function getContacts(Request $request)
    {
        $messanger = new Messenger();
        // get all users that received/sent message from/to
[Auth user]
        $users = Message::join('users',  function ($join) {
            $join->on('ch_messages.from_id', '=', 'users.id')
                ->orOn('ch_messages.to_id', '=', 'users.id');
        })
```

```
        ->where(function ($q) {
            $q->where('ch_messages.from_id', Auth::user()->id)
            ->orWhere('ch_messages.to_id', Auth::user()->id);
        })
        ->where('users.id','!=',Auth::user()->id)
        ->select('users.*',DB::raw('MAX(ch_messages.created_at)
max_created_at'))
        ->orderBy('max_created_at', 'desc')
        ->groupBy('users.id')
        ->paginate($request->per_page ?? $this->perPage);

        $usersList = $users->items();

        if (count($usersList) > 0) {
            $contacts = '';
            foreach ($usersList as $user) {
                $contacts .= $messanger->getContactItem($user);
            }
        } else {
            $contacts = '<p class="message-hint center-
el"><span>Your contact list is empty</span></p>';
        }

        return Inertia::render([
            'contacts' => $contacts,
            'total' => $users->total() ?? 0,
            'last_page' => $users->lastPage() ?? 1,
        ]);
    }


    /**
     * Update user's list item data
     *
     * @param Request $request
```

```
     * @return JsonResponse
     */
    public function updateContactItem(Request $request)
    {
        // Get user data
        $messanger = new Messenger();
        $user = User::where('id', $request['user_id'])->first();
        if(!$user){
            return Inertia::render([
                'message' => 'User not found!',
            ]);
        }
        $contactItem = $messanger->getContactItem($user);

        // send the response
        return Inertia::render([
            'contactItem' => $contactItem,
        ]);
    }

     /**
     * Search in messenger
     *
     * @param Request $request
     * @return JsonResponse|void
     */
    public function search(Request $request)
    {
        $getRecords = null;
        $messanger = new Messenger();
        $input = trim(filter_var($request['input']));
        $records = User::where('id','!=',Auth::user()->id)
                    ->where('name', 'LIKE', "%{$input}%")
                    ->paginate($request->per_page ?? $this-
>perPage);
```

```
        foreach ($records->items() as $record) {
            $getRecords .= view('Chatify::layouts.listItem', [
                'get' => 'search_item',
                'user' => $messanger->getUserWithAvatar($record),
            ])->render();
        }
        if($records->total() < 1){
            $getRecords = '<p class="message-hint center-el"><span>Nothing to show.</span></p>';
        }
        // send the response
        return Inertia::render([
            'records' => $getRecords,
            'total' => $records->total(),
            'last_page' => $records->lastPage()
        ]);
    }

     /**
     * Delete conversation
     *
     * @param Request $request
     * @return JsonResponse
     */
    public function deleteConversation(Request $request)
    {
        // delete
        $messanger = new Messenger();
        $delete = $messanger->deleteConversation($request['id']);

        // send the response
        return Inertia::render([
            'deleted' => $delete ? 1 : 0,
```

```
        ]);
    }

     /**
     * Delete message
     *
     * @param Request $request
     * @return JsonResponse
     */
    public function deleteMessage(Request $request)
    {
        // delete
        $messanger = new Messenger();
        $delete = $messanger->deleteMessage($request['id']);

        // send the response
        return Inertia::render([
            'deleted' => $delete ? 1 : 0,
        ]);
    }

    /**
     * Set user's active status
     *
     * @param Request $request
     * @return JsonResponse
     */
    public function setActiveStatus(Request $request)
    {
        $activeStatus = $request['status'] > 0 ? 1 : 0;
        $status = User::where('id', Auth::user()->id)-
>update(['active_status' => $activeStatus]);
        return Inertia::render([
            'status' => $status,
        ]);
```

```
    }
}
```

В приведенном выше коде была создана функция службы обмена сообщениями и показан код контроллера Messenger, чтобы упростить отправку, удаление и извлечение сообщений в режиме реального времени. Для этой цели использованы веб-сокеты Laravel для мгновенной отправки сообщений как клиентам, так и фрилансерам. Laravel WebSockets обеспечивает связь между сервером и клиентами в режиме реального времени, обеспечивая мгновенное обновление и взаимодействие без необходимости постоянного опроса или обновления страницы. Laravel WebSockets легко интегрируется с системой трансляции событий Laravel, позволяя разработчикам транслировать события по каналам WebSocket подписанным клиентам.

5.2 Система кошелька

```
 function UpdateUserBalance($amount, $operation, $user_id,
$remarks = null)
    {
        try {
            $user = User::findOrFail($user_id);
            if ($operation == 'add') {
                $user->increaseBalance($amount * 100, [
                    'description' => $remarks,
                ]);

                $deposite_amout = $amount * 100;

                $user->deposit($deposite_amout);
            } else if ($operation == 'deduct') {
                $user->decreaseBalance($amount * 100, [
                    'description' => $remarks,
                ]);
```

```
                $deposite_amout = $amount * 100;

                $user->withdraw($deposite_amout);

            } else if ($operation == 'adjust') {
                $reset_amount = $amount * 100 - $user->balance *
100;
                $user->modifyBalance($reset_amount, [
                    'description' => $remarks . ' (Adjusted)',
                ]);
            } else if ($operation == 'reset') {
                $user->resetBalance();
            }
            return true;
        } catch (Exception $e) {
            return false;
        }
    }
}
    /**
     * Reset the balance to 0 or set a new value.
     *
     * @param  int|null $newAmount
     * @param  array $parameters
     *
     */
    public function resetBalance(int $newAmount = null,
$parameters = [])
    {
        $this->balanceHistory()->delete();

        if (is_null($newAmount)) {
            return true;
        }
```

```
        return $this->createBalanceHistory($newAmount,
$parameters);
    }

 /**
     * Increase the balance amount.
     *
     * @param  int $amount
     * @param  array $parameters
     *
     */
    public function increaseBalance(int $amount, array
$parameters = [])
    {
        return $this->createBalanceHistory($amount,
$parameters);
    }

    /**
     * Decrease the balance amount
     *
     * @param  int $amount
     * @param  array $parameters
     *
     */
    public function decreaseBalance(int $amount, array
$parameters = [])
    {
        return $this->createBalanceHistory(-1 * abs($amount),
$parameters);
    }

    /**
     * Modify the balance sheet with the given value.
     *
```

```
     * @param  int $amount
     * @param  array $parameters
     *
     */
    public function modifyBalance(int $amount, array $parameters
= [])
    {
        return $this->createBalanceHistory($amount,
$parameters);
    }
```

В приведенном выше коде были созданы вспомогательные функции и внедрены четыре функции-признака в систему, чтобы повысить надежность системы кошельков. Эти функции вызываются, когда пользователь пополняет баланс с помощью платежного шлюза и когда пользователь отправляет сообщение на новое задание. Это относится только к первому сообщению, отправленному на новое задание.

5.3 Динамическое управление блогами

```
 public function blogs()
    {
        $blogs = Blog::where('status', 1)->paginate(9);
        return Inertia::render('Frontend/Pages/Blogs', [
            'blogs' => $blogs,
        ]);
    }
```

```
import { ref } from 'vue';
import { Head, Link, useForm } from '@inertiajs/vue3';
import Header from '@/Pages/Frontend/Header.vue';
import Footer from '@/Pages/Frontend/Footer.vue';
import SubHeader from
'@/Pages/Frontend/Ankcomponents/SubHeader.vue';
```

```
import AllContact from
'@/Pages/Frontend/Ankcomponents/AllContact.vue';
import Spaces from '@/Pages/Frontend/Ankcomponents/Spaces.vue';
import Testimonial from
'@/Pages/Frontend/Ankcomponents/Testimonial.vue';
import CardBox from "@/Components/CardBox.vue"
import FormField from '@/Components/FormField.vue'
import FormControl from '@/Components/FormControl.vue'
import Pagination from '@/Components/Admin/Pagination.vue';
const props = defineProps({
    blogs: {
        type: Object,
        default: () => ({}),
    },
});
</script>

<template>
    <div>
        <Header />

        <div class="whatweget container mt-[80px]">
            <div class="whatweget-wrap">
                <div class="title flex justify-between  gap-2
items-center">
                    <div>
                        <h2
                            class="lg:text-[48px] text-[32px]
lg:leading-[57.6px] leading-[43.2px] lg:text-center font-bold">
                            Blogs& Events
                        </h2>
                    </div>
                </div>
                <div class="get-body mt-[50px]">
```

```
                    <div class="news-wrap grid lg:grid-cols-3
sm:grid-cols-2 grid-cols-1 gap-[20px] h-full">

                        <div v-for="newa in blogs.data"
class="news-tile">
                            <Link
:href="route('blogDetail',newa.slug)">
                            <div class="news-tile-wrap">
                                <div class="top-news h-[250px]">
                                    <img class="rounded-[10px]
aspect-video h-full object-cover"
                                        :src="'/storage/' +
newa.image" alt="" />
                                </div>
                                <div class="bottom mt-[32px]">
                                    <h2 class="font-bold text-
[24px] leading-[28.8px]">{{ newa.name }}</h2>
                                    <h3 class="text-base mt-
[8px] !line-clamp-2" v-html="newa.description"></h3>
                                    <span class="mt-[24px] text-
textcolor text-[14px] leading-[21px]">{{ new

Date(newa.created_at).toLocaleString() }}</span>
                                </div>
                            </div>
                            </Link>
                        </div>

                    </div>

                    <div class="mt-[20px] mb-[20px]">
                        <Pagination :data="blogs" />
                    </div>
                </div>
            </div>
```

```
        </div>

        <Footer />
    </div>
</template>
```

5.4 Система управления вопросами

Эта система является наиболее важной частью проекта, и выполнить ее было очень сложно. Администратор имеет доступ к этой системе. Администратор может создавать вопросы любого типа для любых проектов. Администратор полностью управляет системой управления вопросами для каждого проекта.

```
namespace App\Http\Controllers\Question;

use App\Http\Controllers\Controller;
use App\Models\Category\Category;
use App\Models\Question\Question;
use App\Models\Question\QuestionCatgory;
use App\Models\Question\QuestionOption;
use Exception;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use Inertia\Inertia;

class QuestionController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $questions = (new Question())->newQuery();

        if (request()->has('search')) {
```

```
            $questions->where('question', 'Like', '%' .
request()->input('search') . '%');
        }

        if (request()->query('sort')) {
            $attribute = request()->query('sort');
            $sort_order = 'ASC';
            if (strncmp($attribute, '-', 1) === 0) {
                $sort_order = 'DESC';
                $attribute = substr($attribute, 1);
            }
            $questions->orderBy($attribute, $sort_order);
        } else {
            $questions->latest();
        }

        $questions = $questions-
>paginate(config('admin.paginate.per_page'))
            ->onEachSide(config('admin.paginate.each_side'))
            ->appends(request()->query());

        return Inertia::render('Admin/Question/Index', [
            'questions' => $questions,
            'filters' => request()->all('search')
        ]);
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        $category = Category::where('status', 1)->pluck('name',
'id');
        return Inertia::render('Admin/Question/Create', [
```

```
            'category' => $category,
        ]);
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        $validate = $request->validate([
            'question' => 'required',
            'input_type' => 'required',
            'is_required' => 'nullable',
            'placeholder' => 'nullable',
            'input_text' => 'nullable',
            'label' => 'required',
            'category_id' => 'required',
            'status' => 'nullable',
        ]);

        try {
            DB::beginTransaction();
            $question = Question::create([
                'question' => $request->question,
                'input_type' => $request->input_type,
                'is_required' => $request->is_required,
                'placeholder' => $request->placeholder,
                'input_text' => $request->input_text,
                'input_id' => rand(0000, 9999),
                'status' => $request->status ? $request->status
: 0,
                'label' => $request->label,
            ]);

            if ($question) {
```

```
                $question_category = QuestionCatgory::create([
                    'question_id' => $question->id,
                    'category_id' => $request->category_id,
                ]);
                if ($question->input_type == 'checkbox' ||
$question->input_type == 'radio' && $request->options) {
                    foreach ($request->options as $option_value)
{
                        $question_options =
QuestionOption::create([
                            'question_id' => $question->id,
                            'name' => $option_value['name'],
                            'value' => $option_value['value'],
                        ]);
                    }
                }
            }

            DB::commit();

            if ($question) {
                return redirect()-
>route('admin.question.index')->with('message', 'Question create
successfully');
            } else {
                return redirect()-
>route('admin.question.index')->with('error', 'Something went
wrong.');
            }
        } catch (Exception $e) {

            return redirect()->route('admin.question.index')-
>with('error', 'Something went wrong.');
        }
    }
```

```
    /**
     * Display the specified resource.
     */
    public function show(string $id)
    {
        $question = Question::where('id', $id)-
>with('questionOptions', 'questionCategory')->firstOrFail();
        $category = Category::where('status', 1)->pluck('name',
'id');
        return Inertia::render('Admin/Question/Show', [
            'question' => $question,
            'category' => $category
        ]);
    }

    /**
     * Show the form for editing the specified resource.
     */
    public function edit(string $id)
    {
        $question = Question::where('id', $id)-
>with('questionOptions', 'questionCategory')->firstOrFail();
        $category = Category::where('status', 1)->pluck('name',
'id');
        $QuestionOption = QuestionOption::where('question_id',
$question->id)->get();
        return Inertia::render('Admin/Question/Edit', [
            'category' => $category,
            'question' => $question,
            'QuestionOption' => $QuestionOption,
        ]);
    }

    /**
```

```
     * Update the specified resource in storage.
     */
    public function update(Request $request, string $id)
    {
        $validate = $request->validate([
            'question' => 'required',
            'input_type' => 'required',
            'is_required' => 'nullable',
            'placeholder' => 'nullable',
            'input_text' => 'nullable',
            'label' => 'required',
            'category_id' => 'required',
            'status' => 'nullable',
        ]);

        try {
            DB::beginTransaction();

            $question = Question::where('id', $id)-
>firstOrFail();


            $question->update([
                'question' => $request->question,
                'input_type' => $request->input_type,
                'is_required' => $request->is_required,
                'placeholder' => $request->placeholder,
                'input_text' => $request->input_text,
                'status' => $request->status ? $request->status
: 0,
                'label' => $request->label,
            ]);

            if ($question) {
```

```
                $question_category =
QuestionCatgory::where('question_id', $question->id)->first();

                if ($question_category) {
                    $question_category->update([
                        'category_id' => $request->category_id,
                    ]);
                }
                if ($question->input_type == 'checkbox' ||
$question->input_type == 'radio' && $request->options) {
                    foreach ($request->options as $option_value)
{
                        if (isset($option_value['id'])) {
                            $QuestionOption =
QuestionOption::where('id', $option_value['id'])->first();
                            if ($QuestionOption) {
                                $QuestionOption->update([
                                    'name' =>
$option_value['name'],
                                    'value' =>
$option_value['value'],
                                ]);
                            }
                        } else {
                            $question_options =
QuestionOption::create([
                                'question_id' => $question->id,
                                'name' => $option_value['name'],
                                'value' =>
$option_value['value'],
                            ]);
                        }
                    }
                }
            }
```

```
            DB::commit();

            return redirect()->route('admin.question.index')
                ->with('message', __('Question updated
successfully.'));
        } catch (Exception $e) {

            return redirect()->route('admin.question.index')-
>with('error', 'Something went wrong.');
        }
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(string $id)
    {
        $question = Question::where('id', $id)->firstOrFail();

        $question->delete();

        return redirect()->route('admin.question.index')
            ->with('message', __('Question deleted
successfully'));
    }
}
```

6 Тестирование веб-приложения

6.1 Кросс-браузерное тестирование

Это процесс тестирования веб-сайта в разных браузерах и на разных устройствах. Основная цель тестирования - проверить, не выглядит ли дизайн необычно. это также гарантирует идеальную работу веб-приложения во всех браузерах. Большинство пользователей Интернета используют мобильные устройства. Необходимо провести верификацию мобильной версии веб-сайта. Была проведена верификация на всех устройствах и внесены исправления. В конечном итоге качество веб-приложения было повышено.

6.2 Ручное тестирование

Ручная верификация — это процесс проверки веб-приложения без использования каких-либо инструментов, программного обеспечения или каких-либо других систем. В ходе этого процесса тестирования была проверена работа веб-приложения, которая включает в себя перенаправление веб-страницы при нажатии кнопки. Были обнаружены и исправлены некоторые логические ошибки проектирования. Были исправлены визуальные аспекты, включая шрифты и цвета. В результате был значительно оптимизирован удобный для пользователя характер веб-приложения. Вот некоторые основные факторы, влияющие на ручную проверку:

Гибкость и адаптивность: быстро внес изменения в приложение, что сократило время на создание автоматизированных тестов.

Проверка пользовательского опыта: ручное тестирование помогло проанализировать скорость и удобство использования веб-приложения с точки зрения пользователя. В ходе проверки были выявлены проблемы с навигацией, нерабочие ссылки для перенаправления, которые позже были исправлены.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы был выполнен каждый важный шаг. Это включало тщательное изучение рынка и углубленный анализ основных конкурентов в отрасли, с которыми будет конкурировать разрабатываемая система. Детальная архитектура интерфейса была тщательно проработана с учетом всех нюансов и тонкостей продуктового рынка, а серверная часть построена с использованием современного фреймворка Laravel 11(5). Обширное тестирование проводилось в несколько этапов для выявления и устранения многочисленных ошибок, что в конечном итоге привело к успешному завершению архитектуры программирования на весьма продвинутом уровне. Основная цель внедрения последней версии Laravel 11(5) заключалась в эффективном управлении системой чата без необходимости использования технологии WebSocket. После тщательного изучения документации официального веб-сайта Laravel было установлено, что использование этой платформы гарантирует бесперебойную работу системы чата. Внедренная система отзывов позволяет пользователям оценивать как клиентов, так и фрилансеров, повышая доверие к платформе. Проект также отдает приоритет удобному для пользователя дизайну веб-сайта и эффективному управлению блогом. Кроме того, была включена функция фильтра фрилансеров, чтобы упростить процесс поиска проектов и сэкономить время пользователей. Поэтому разработанную систему можно сравнить с такими известными платформами, как profi.ru, Upwork и Fiverr. Он был разработан для облегчения беспрепятственного общения между клиентами и фрилансерами и предлагает широкий спектр инструментов для поддержки эффективного сотрудничества.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) Laravel Documentation [Электронный ресурс]. – URL: https://laravel.com/docs/11.x/readme (Дата обращения: 11.04.2023).

2) Upwork: Connecting Clients and Freelancers [Электронный ресурс]. – URL: https://www.upwork.com/about (Дата обращения: 11.04.2023).

3) Fiverr: Freelance Services Marketplace [Электронный ресурс]. – URL: https://www.fiverr.com/about (Дата обращения: 11.04.2023).

4) Profi.ru: Leading Freelance Platform in Russia [Электронный ресурс]. – URL: https://profi.ru/about (Дата обращения: 11.04.2023).

5) Advantages of Laravel for Web Development [Электронный ресурс]. – URL: https://laravel-news.com/advantages-of-laravel (Дата обращения: 11.04.2023).

6) Laravel Security Features [Электронный ресурс]. – URL: https://laravel.com/docs/11.x/security (Дата обращения: 11.04.2023).

7) Laravel and WebSockets [Электронный ресурс]. – URL: https://laravel.com/docs/11.x/broadcasting (Дата обращения: 11.04.2023).

8) How Laravel Eases API Development [Электронный ресурс]. – URL: https://laravel-news.com/api-development-laravel (Дата обращения: 11.04.2023).

9) Web Development with Laravel [Электронный ресурс]. – URL: https://books.google.com/web-development-with-laravel (Дата обращения: 11.04.2023).

10) Market Analysis for Freelance Platforms [Электронный ресурс]. – URL: https://www.freelanceplatforms.com/market-analysis (Дата обращения: 11.04.2023).

11) Building Secure Web Applications with Laravelt [Электронный ресурс]. – URL: https://security.laravel.com/building-secure-web-apps(Дата обращения: 11.04.2023).

12) Trends in Freelance Work [Электронный ресурс]. – URL: https://www.freelancetrends.com/trends-in-freelance-work (Дата обращения: 11.04.2023).